Main Program and System Scheduling

Meituan Challenge Robot Code Guide 01 | English version | 2026-10-13

1 Overview

This document explains the ABB RAPID module `Modul1` in `Modul.docx`. It defines shared data, taught robot targets, and runtime states, and repeatedly invokes the battery task through `main`. The companion `Battery` module handles vision-guided picking and placing, while `Connect_socket` handles communication with the camera and PLC. The three guides cover scheduling, task execution, and communication for the same source version and should be read together.

The current program combines vision-based localization with predefined motion and sequential control. The supplied material contains no implementation of VLA, Diffusion Policy, model training, autonomous navigation, or complete battery-swapping mechanism control. This guide does not present those capabilities as implemented features.

2 Module Responsibilities and Call Relationships

Source file and module	Main responsibilities	Key entry points
Modul.docx / Modul1	Shared variables, taught targets, main loop, and task scheduling	main, Task_Set_2
battery.docx / Battery	Obtain vision coordinates, select a task, and execute picking and placing	Battery_work
connect_socket.docx / Connect_socket	Establish connections, send and receive strings, parse coordinates, and handle PLC messages	Connect2, ProcessReceivedData

The main call chain is shown below. `initial` is an external dependency; its definition was not found in any of the three supplied modules.

```
main
  -> initial [definition not included in the three documents]
  -> StartProgram := TRUE
  -> WHILE StartProgram
       -> Task_Set_2
            -> Battery_work
                 -> Battery_get_values
                      -> Connect2
                           -> SocketConnectCamara
                                -> ProcessReceivedData
                 -> Battery_work_2 or Battery_work_1
```

`Task_Set_2` is the name of a scheduling procedure; it does not mean that the main program executes only vision Task 2. The actual selection between single-battery and multiple-battery operations takes place in `Battery_work`.

3 Main Program Execution Flow

1. `main` first calls `initial`, then waits for 0.1 seconds. Because the initialization procedure is not included in the material, its specific actions and initial states cannot be determined from the current files.

2. The program sets `StartProgram` to `TRUE` and enters the `WHILE StartProgram DO` loop.

3. Each iteration executes `Task_Set_2`. This procedure prints a start message, calls `Battery_work`, and prints a finish message after it returns.

4. The three current modules contain no observed statement that sets `StartProgram` to `FALSE` when a task ends normally, so the main loop proceeds to the next iteration.

“Finish Task” indicates that the called procedure has returned. It is not a detection result confirming that a battery was successfully picked, correctly placed, or that the competition task was completed. An early return due to no target may also lead to this message.

4 Data and Equipment Dependencies

Data category	Representative identifiers	Role in the execution chain
Communication data	socket1, socket2, input_string, send_string, receive_string_copy	Distinguish the camera and PLC channels and carry requests and replies
Vision results	nval1 through nval6	Store parsed values for a single target or three target groups
Task and communication states	StartProgram, socket_start, startPLC, CamaraFinish	Control the main loop, select communication branches, and mark the end of parsing
Vision offsets	diamondX, diamondY	Add common position offsets after coordinates are obtained
Robot targets	Redy_Camara_Diamonds, D_pick, D_put_Red/Blue/Green/Yellow	Provide a camera reference pose, a pick-height reference, and placement targets
Tool and workobject coordinates	tool_D, wobj0, wobjDiamond	Specify the tool and reference coordinate systems for motion instructions

`Modul1` also retains multiple targets with W, P, and B prefixes and other state variables. Not all of these definitions are used in the currently visible battery call chain, so their presence does not establish that the corresponding operations will run. Calibrated coordinates are specific to a workstation and should not be copied directly as general-purpose parameters for another robot.

5 Integration and Verification Requirements

The three DOCX files contain code text; they are not a complete, directly deployable RobotStudio station or controller backup. The following conditions must be checked in the target workstation before determining whether the program can run:

1. Supply and verify the actual definition of `initial`, the initialization sequence, and the visibility of shared data across RAPID modules.

2. Check the workstation configuration for `tool_D`, `wobjDiamond`, and digital output `IIO_do2`; these configurations are not fully provided in the three documents. Confirm the actual reference frame represented by `wobj0`.

3. Check consistency between camera coordinates, `diamondX/diamondY`, tool orientation, pick height, and taught placement targets.

4. Check syntax and dependencies in the corresponding controller or simulation environment, then have on-site personnel perform controlled single-step and low-speed verification.

5. Verify execution behavior separately for no target, a single red target, three-color targets, communication failures, and repeated loop iterations.

This guide provides a static interpretation of the code. No compilation records, simulation results, physical robot logs, or success-rate statistics accompany the material. It therefore makes no claim that the program has compiled successfully, passed physical robot verification, or achieved any performance metric.

6 Source References

Source version: `lkkkkk667/meituan`, commit `68e262bbedcf2269ba3a7321a045f97e96f185b6`.

- `Modul.docx`: variable and target definitions under `MODULE Modul1`; `PROC main()`; `PROC Task_Set_2()`.

- `battery.docx`: `PROC Battery_work()` and `PROC Battery_get_values()`.

- `connect_socket.docx`: `PROC Connect2()`, `PROC SocketConnectCamara()`, and `PROC ProcessReceivedData()`.

Related guides: Battery Task · Communication Module.